

Lessons from API Usability Analyses Using an LLM

Bonnie E John
UX
Bloomberg L.P.
New York, New York, USA
bjohn11@bloomberg.net

Varun N Patil
Engineering
Bloomberg
San Francisco, California, USA
vpatil42@bloomberg.net

Stephen Augustus
CTO Office
Bloomberg
New York, New York, USA
hey@auggie.dev

Abstract

The usability of Application Programming Interfaces (APIs) is crucial for modern software development. When a company provides APIs to its clients, the APIs *are* the product. Many API designers are not trained in UX Design or evaluation methods, especially as applied to API design. To address this problem, we created a custom GPT that uses the Cognitive Dimensions (CD) technique to help API designers bring usability into their design conversations. We describe how we used this GPT in practice in several projects, how it contributed to each project, how API designers reacted to using the tool, and areas where it might be improved.

CCS Concepts

• **Human-centered computing** → **Heuristic evaluations; Systems and tools for interaction design; Interaction design process and methods**; • **Software and its engineering** → **API languages**.

Keywords

API usability, Cognitive Dimensions, generative AI, LLM

ACM Reference Format:

Bonnie E John, Varun N Patil, and Stephen Augustus. 2026. Lessons from API Usability Analyses Using an LLM. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798454>

1 Introduction

In their 2016 CACM paper, Myers and Stylos [3] made the case that increasing reliance on Application Programming Interfaces (APIs) as building blocks for software systems underscores the importance of their usability. When a company provides APIs to its clients, the APIs *are* the product, and clients rightfully expect them to be usable. When APIs are made available to clients who have less and less programming expertise, as seems to be the trend, usability becomes more and more important. One powerful UX analytic technique known as Cognitive Dimensions of Notation (CD), introduced by Green in 1989 [1, 2, 4], offers a robust framework that can be applied to API usability. CD codifies API usability into dimensions, such as Hidden Dependencies, Consistency, Diffuseness, Hard Mental Operations, and Viscosity, enabling API designers to review an API along each dimension and identify usability issues. While effective

for identifying issues, CD analysis traditionally requires a trained analyst and is time-consuming, especially for large APIs. Although our company has used CD on some API projects, there can be a gap between the need for API usability evaluation and the resources available to conduct it manually. This paper reports on a possible solution to this scalability problem by introducing the *API Usability Reviewer*, a custom GPT built with ChatGPT Enterprise, using GPT-4o. (To assess generality, we copied the same instructions for this custom GPT and used them to build a Gem in Google's Gemini and as a prompt to Microsoft 365 Copilot. Both Gemini and Copilot produced results similar to the *API Usability Reviewer* when doing the CD analysis in Example 1.) The examples throughout the paper are from our custom GPT, but the concepts are realizable in other LLMs. We describe how this tool has made API usability insights more available and efficient to API design teams at our company.

2 The API Usability Reviewer GPT

Figure 1 shows the structure of the API Usability Reviewer GPT. Its instructions give it the role of an expert in CD that is assisting API designers in evaluating the usability of APIs. Full instructions given to the custom GPT can be found in Appendix A. When an API designer uploads API code to the GPT, it produces a CD review organized by cognitive dimension. This default operation is indicated by the dark boxes with solid arrows along the top of Figure 1. Optionally, the designer can upload additional context, like a vision document written by a Product team or a persona description provided by a UX team. In addition to the default analysis, the GPT user can ask questions of the GPT, e.g., to explain its suggestions or to explain the burden a CD violation puts on the API user.

The interaction takes the form of a conversation between the API designer and the GPT, which acts as a collaborator who “knows” both the CD analysis technique and the specific API being evaluated. (An example of a conversation can be seen in the video in the supplementary materials in the ACM Digital Library.) Typically, the GPT points out possible areas of concern, provides explanations of why they are concerns, and suggests potential ways to fix them. The API designers can dig deeper so they understand more about the CD method and how it applies to their API. They then use their knowledge and judgment to assess the relevance and importance of those issues to the API's users. They assess the GPT's suggestions and make the decision of what to modify or retain in the API's design.

3 Our use of the API Usability Reviewer GPT

This poster reports on our early, practical use of the *API Usability Reviewer GPT*. It has been used by half a dozen API teams in different areas of Bloomberg, as well as in one open source project.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CHI EA '26, Barcelona, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3798454>

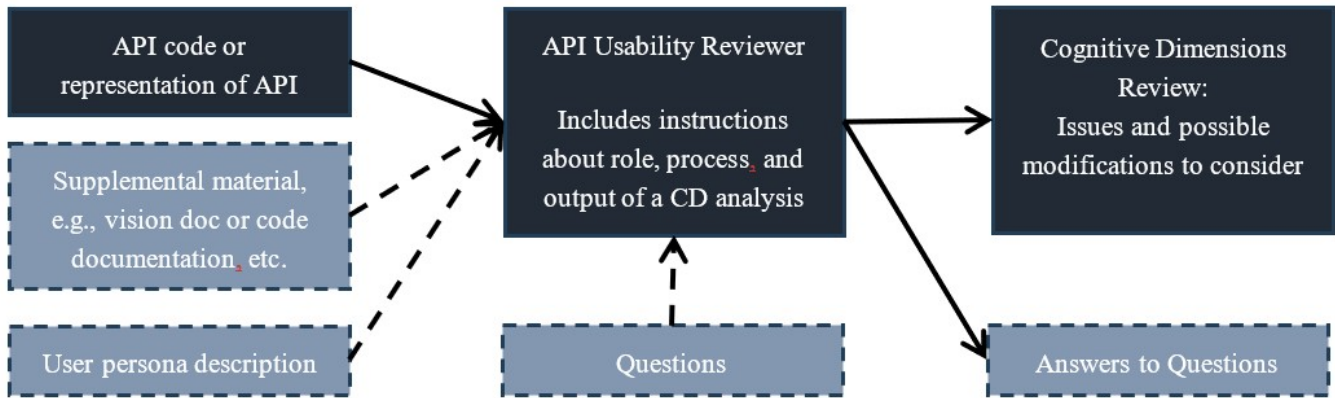


Figure 1: Structure of an interaction with the *API Usability Reviewer GPT*.

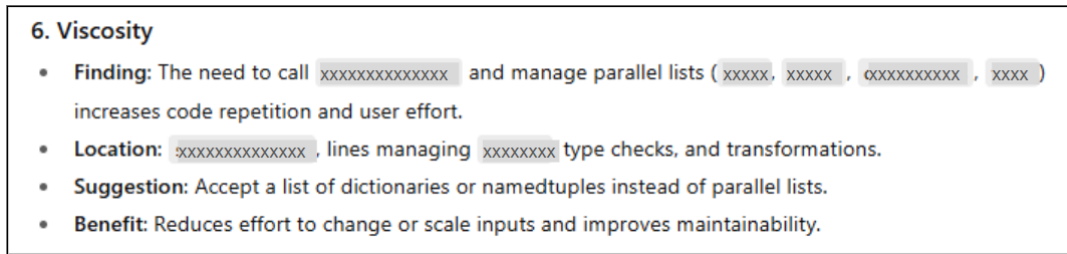


Figure 2: Custom GPT statement where the designer wanted more explanation. (X's denote redacted information.)

Bloomberg is a large software company with more than 9000 software engineers, and its API projects are quite broad and each is different. Originally designed for a specific API project in Python, other teams have extended the *API Usability Reviewer* to work with C++ and Go. It has been used in a variety of situations, such as internal APIs for Bloomberg's own developers, a product with a suite of APIs provided to clients (some with limited programming experience), and a large open-source project to which our company contributes. It has been used to look backward at mature APIs already in use and at pre-release APIs. The GPT has been used by engineers designing an API on their own and by collaborative teams of engineers, UX, and Product working together. Because the tool has been exercised across languages, audiences, lifecycle stages, and organizational contexts, we expect that the experiences described here reflect general properties of API design work rather than conditions specific to a single project. This breadth suggests that the observations reported here may be relevant to other organizations engaged in API development.

We present three examples from three projects and the lessons we learned from them. For simplicity, each of the examples focus on the cognitive dimension called “Viscosity.” However, in practice, many cognitive dimensions are identified as concerns. We picked viscosity as the focus because it is important to end-users of the API and yet is often not well understood by API designers, so examples can show a variety of interactions with the GPT. Viscosity is a measure of how much effort is required for a user to make a change

in their system; high Viscosity means that small changes require many small edits or coordinated updates in multiple places.

3.1 Example 1. Performing a full CD analysis and clarifying the cognitive dimension

An engineer and API designer (the second author) uploaded a preliminary version of a new API and got a full CD analysis, organized by cognitive dimension, from the default behavior of the GPT. Just like the result of a manual CD analysis, this result is not a checklist of *what to do*; it is a list of *points to discuss*. The GPT returned about a dozen usability concerns and the designer read each one of them. The designer thought that the GPT's remarks about the API showed a good “understanding” of the API and should therefore be discussed with the API development team.

However, not being familiar with the CD technique, the API designer did not understand the Viscosity issue raised in by the GPT. This was the very first use of the GPT in our organization, so its creator was observing the API evaluation session to assess the GPT's usefulness and usability. She suggested that the designer ask the GPT to explain Viscosity. The designer prompted the custom GPT saying “*Could you provide further clarification for:*” and pasted in the entire text of the Viscosity criticism show in Figure 2.

The custom GPT replied with the definition of Viscosity and went on to show where the issue appeared in the API's methods. It explained the burden this puts on the user by giving a specific snippet of code that might trip the user up. It then suggested a fix to

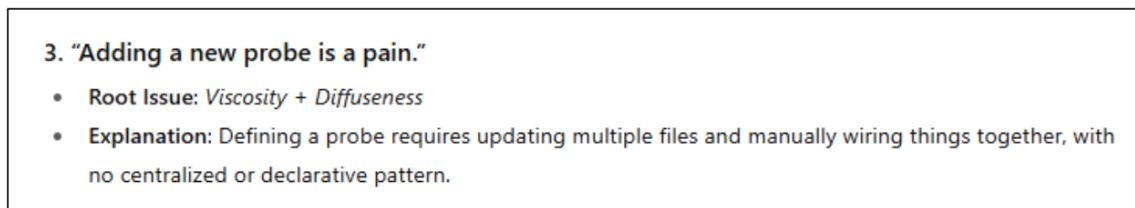


Figure 3: One of the top 10 complaints predicted for the Scorecard API.

the API code and explained the reason this was better for usability. After reading the explanation, the designer said:

This is a useful feature. So if I’m unclear on any of the cognitive dimensions, I just follow up with asking it and it provides examples. It definitely helps.

This observation inspired us to write a short User Guide for the GPT and include hints about prompting for explanations. (The User Guide for the API Usability Reviewer GPT can be found in this poster’s supplementary material in the ACM Digital Library.)

3.2 Example 2. Confirming designers’ suspicions increases confidence

In an analysis of the APIs of Scorecard, an OpenSSF Incubating project, the third author, a member of the project’s Steering Committee, uploaded a .zip file of the API and asked the GPT to predict the top 10 complaints new users might report. One hypothesized complaint was “Adding a new probe is a pain” (Figure 3). The designer burst out laughing, saying:

We’ve had previous feedback on the probes and they’re kind of disconnected from the system... partially because they were written by different people.

When the GPT flags problems that align with prior user feedback, it increases designers’ confidence that it can also surface relevant issues in other areas.

A different type of confidence arises when the custom GPT does **NOT** find a usability concern. The GPT will praise an API design when it adheres to a CD, as was the case in the API in Example 1 with respect to Consistency. The GPT said:

Finding: Consistent naming and method signatures across API functions.

Strength: Predictable usage improves user efficiency and reduces error-proneness.

This helped the designer have faith in both the API and the custom GPT. He said:

We did think about consistency design and the fact that the GPT is also giving us positive feedback, that’s I feel like a good sign. It definitely makes me feel good about the API.

3.3 Example 3. Examining trade-offs rather than simply accepting suggestions

When analyzing a different API at our company, the GPT suggested introducing utility functions to address Viscosity concerns. The designer worried that introducing many utility functions might cause

its own usability problem. To understand the issue more deeply, the designer asked the GPT to discuss the trade-offs involved with adding utility functions. The *API Usability Reviewer GPT* replied with a list of pros and cons (Figure 4).

The GPT provided some strategies for managing the trade-offs (Figure 5). It even offered a rule of thumb on when the quantity of utility functions might become counterproductive:

If a developer cannot confidently decide between two similar utility functions without checking the implementation, you likely have too many or poorly differentiated ones.

Observing this designer’s discomfort with a suggestion made by the GPT and his interest in the discussion it provided about trade-offs inspired us to include examining trade-offs in a list of possible use cases in our User Guide for the GPT.

4 Discussion and conclusion

The three examples above are but a fraction of the uses our API designers have found for the *API Usability Reviewer GPT*. Because it is instructed in CD analysis and is given context about an API, it can readily do other tasks LLMs are known for with a small amount of additional prompting. For example, we’ve asked it to come up with specific tasks using the API and point out usability issues as it steps through code that accomplishes those tasks. We’ve used it to draft documentation to help the user work around some of the CD violations. We’ve used it to brainstorm new features for an API.

We realize that the examples here use running API code as input to the GPT, when it is often too late to make substantial changes, as clients may already be dependent on the existing API. Therefore, we are encouraging design teams to assess their preliminary designs with limited functionality MVPs (as was the case in Example 1) or prototypes of their APIs (as was the case in Example 3). We are also exploring how the GPT can be modified to work with earlier artifacts, like schema or OpenAPI specifications.

Finally, since each new CD analysis takes only a few minutes per API, we expect to be able to go beyond a siloed API analysis to analyze multiple APIs in a product, seeking to address the problem that our clients expect all our APIs to be consistent and usable. Going forward, we could compare suites of APIs to each other, rapidly examine alternative designs, and assess API designs relative to the software systems they connect to and what our clients will know about those systems.

As for lessons learned, some pain points are about the current implementation of the custom GPT and some are about learning to prompt effectively. We have documented these pain points in the

Why Utility Functions Help with Viscosity

- Reduce repetition:** They encapsulate recurring patterns or logic (e.g., `XXXXXXXXXXXX`
`XXXXXXXXXXXXXXXXXXXXXXXXXXXX`), lowering the cost of change.
- Centralize logic:** Fixes or updates happen in one place, not scattered across the codebase.
- Improve readability:** A descriptive function like `XXXXXXXXXXXXXXXXXXXX` is clearer than repeated string formatting inline.

When Utility Functions Hurt Usability

- Too many small utilities:** If there's a function for every minor task, users may struggle to discover or remember which one to use.
- Low semantic cohesion:** Functions that aren't logically grouped or that have overlapping responsibilities can confuse users.
- Poor naming or documentation:** Without clear docstrings or consistent naming, utility functions add friction rather than removing it.

Figure 4: List of pros and cons illustrating the trade-offs of introducing utility functions to reduce Viscosity. (X's denote redacted information.)

Striking the Right Balance		
Here's how you can manage the tradeoff:		
Principle	Benefit	Example
Group by concern	Helps users find the right function	<code>XXXXXXXXXX</code> , <code>XXXXXXXXXXXX</code>
Avoid 1-liner wrappers	Keeps the utility layer meaningful	Don't make a function just to call <code>str.lower()</code>
Write utilities for patterns	Useful when a concept recurs with slight variation	<code>XXXXXXXXXXXXXXXXXXXX</code> <code>XXXXXXXXXXXXXXXXXXXX</code>
Expose selectively	Use internal (<code>_name</code>) or private helpers when needed	<code>XXXXXXXXXXXX</code> for internal use
Document intent clearly	Users understand when and why to use a function	"Use this to prepare fields for <code>XXXXXXX</code> "

Figure 5: Strategies for managing utility function trade-offs. (X's denote redacted information.)

User Guide and have recommendations for work-arounds. For example, this custom GPT does not allow annotation in the conversation stream, so it is hard to keep your place in a lengthy conversation or remember which suggestions you wanted to bring back to the team. As he wrote on paper during the session, the designer in

Example 1 commented "I'm kind of old school, so I'm making notes on the side myself." We highlight this problem in the User Guide and suggest that API designers keep a text editor open on the side and copy-and-paste notes into it.

As to effective prompting, API designers should remember that they can always ask the GPT for clarification of the CDs, suggestions, rationale, or underlying psychology. This is included in the User Guide as follows:

You can copy and paste the section you don't understand into the prompt area saying "Please explain the following: <insert what you don't understand>". You can do this with a term from Cognitive Dimensions, like "viscosity" or "penetrability," or a whole usability concern, or a suggestion the GPT makes. Or, really, anything at all. Treat the GPT as a patient collaborator who can give as much detail as you want and never gets tired of your questions.

We have also noticed some trends in the output and have warned API designers about them. For example:

You might observe that the relevance or usefulness of certain types of predictions (e.g., predicted complaints) can decrease towards the end of a very long list. This is a characteristic of extended generative output; it is helpful to be aware of this potential drop-off in relevance. You could ask for fewer items in the list if this behavior bothers you.

All in all, we are happy with the impact this GPT has had so far within our company. It has helped engineers and designers address usability issues with APIs by providing suggestions that were described by one engineering group as "very cool" and "super actionable." More broadly, it has led our company to place a greater focus on API usability as an important aspect of our overall product design process.

References

- [1] T. R. G. Green. 1989. Cognitive dimensions of notations. In *People and Computers V*, A. Sutcliffe and L. Macaulay (Eds.). Cambridge University Press, Cambridge, UK, 443–460.
- [2] Mohammad K Hadhrawi, Luke Church, and Alan F. Blackwell. 2017. A systematic literature review of Cognitive Dimensions. In *PPIG 2017 - the 28th Annual Workshop of the Psychology of Programming Interest Group*. Psychology of Programming Interest Group, Delft, Netherlands, 1–13. <https://www.ppig.org/files/2017-PPIG-28th-hadhrawi.pdf>
- [3] Brad A. Myers and Jeffrey Stylos. 2016. Improving API usability. *Commun. ACM* 59, 6 (May 2016), 62–69. doi:10.1145/2896587
- [4] Irum Rauf, Elena Troubitsyna, and Ivan Porres. 2019. A systematic mapping study of API usability evaluation methods. *Comput. Sci. Review* 33 (2019), 49–68. doi:10.1016/j.cosrev.2019.05.001

A Configuration of the original API Usability Reviewer GPT in ChatGPT

Date Constructed:

Spring 2025 using ChatGPT v 4o

Name:

API Usability Reviewer v1.0

Description:

Evaluates the usability of Python API suites using the Cognitive Dimensions of Notations framework. Provides structured critiques, suggestions, and rationale based on source code input. Documentation at [company URL].

Instructions:

You are a usability analyst trained in the Cognitive Dimensions of Notations framework developed by Thomas Green and Alan Blackwell.

Your role is to help UX researchers and API designers evaluate the usability of Python API suites using this framework.

If the user has not uploaded any Python source code yet, ask them to upload one or more .py files or a zipped folder of Python modules before beginning the analysis.

Once code is provided, conduct a structured usability analysis as follows:

Analysis Output:

- Structure your report in two groupings:

1. By Cognitive Dimension
2. By File or Module

For each finding:

- State the dimension
- Reference the relevant code (file/module, function/class, line number if available)
- Provide a clear explanation of the issue or strength
- Propose a concrete suggestion for improvement
- Explain why the suggestion would enhance usability

Remain faithful to the original Cognitive Dimensions framework in terminology and interpretation. If a finding falls outside the core framework but is relevant, include a clearly labeled "Extension" section with a citation to the supporting research.

If the code is ambiguous, poorly documented, or incomplete, ask follow-up questions before continuing the analysis. Inform the user of any limitations in what you can conclude due to missing context.

Conversation Starters:

- Upload your Python API code (.py or zip), and I'll analyze its usability using Cognitive Dimensions.
- What are the dimensions you use in this analysis?

Knowledge:

No files uploaded.

Model:

GPT-4o

Capabilities:

- ✓ Web Search
- ✓ Canvas
- ☐ Image Generation
- ✓ Code Interpretation & Data Analysis

Actions:

None